

To fix a website that "breaks" when using the browser back and forward buttons, you must synchronize your application's internal state with the browser's navigation history. This most often happens in Single Page Applications (SPAs) where content changes without a full page reload. 🌱 Telerik.com +3

1. Synchronize State with the History API [🔗](#)

The most effective way to handle navigation is the **History API**, which allows you to manually add or update history entries without reloading the page. 📖 MDN Web Docs +2

- `pushState()` : Use this method when a user performs an action that should be "rewindable" (like opening a modal or switching a tab). This adds a new entry to the browser's history.
- `replaceState()` : Use this to update the current history entry's state without adding a new one (e.g., updating a search filter).
- `popstate` **Event**: Listen for this event to detect when the user clicks the back or forward buttons. When triggered, use the event's `state` object to restore your application to that specific point. 🍳 Stack Overflow +4

javascript

```
// Example: Listening for back/forward clicks
window.addEventListener('popstate', (event) => {
  if (event.state) {
    restoreAppState(event.state); // Function you define to update UI
  }
});
```

Use code with caution.

2. Manage State Persistence

If your site "breaks" because data is lost upon navigating back, you need to store that data somewhere more permanent than just local variables. 🍳 Stack Overflow

- **URL Query Strings**: Store critical UI state (like filter settings or page numbers) in the URL (e.g., `?page=2`). This ensures the state is preserved even if the user refreshes the page.
- **Web Storage**: Use `sessionStorage` or `localStorage` to save temporary user data that should survive a navigation event.

- **Pageshow Event:** Use the `pageshow` event to detect when a page is restored from the browser's cache (Bfcache). Checking `event.persisted` can tell you if the page was loaded from cache, allowing you to re-run necessary initialization logic.  Medium · Aditya Singh Dabhra +3

3. Prevent Unwanted Navigation

In specific cases, like an active checkout or a timed exam, you may need to restrict navigation. 

- `beforeunload` **Event:** Use this to show a confirmation dialog if the user has unsaved changes. Browsers will show a standard prompt asking if they want to leave.
- **Avoid Redirect Traps:** Do not use "filler" pages that immediately redirect users forward, as this can trap them in a loop when they try to go back.  Reddit · r/explainlikeimfive +2

4. Framework-Specific Solutions

If you are using a modern framework, use its built-in router instead of raw JavaScript to ensure navigation is handled correctly.  Kirill Rakhman

- **React/Next.js:** Use React Navigation or Next.js Router to intercept and manage history state.
- **Angular:** Utilize Router Events to detect navigation changes and react accordingly.  GitHub +1